

### c) Scraping completo de clientes usando el sitemap (con paginación)

Objetivo: descargar la información completa de **todos** los clientes registrados y guardarla en `data/clientes.csv`.

A diferencia de productos, `sitemap-clientes.xml` **no** enumera cada cliente individualmente (no existe una página de detalle pública `/clientes/{id}`, solo un listado en `/clientes` y un formulario en `/clientes/nuevo`; la ruta `/clientes/{id}/editar` existe pero está bloqueada en `robots.txt`). Por eso la estrategia combina dos técnicas:

1. Usar el sitemap para descubrir de forma programática la URL del listado de clientes (en vez de asumirla/escribirla a mano).
2. Recorrer la **paginación** de ese listado (`/clientes?page=1`, `?page=2`, ...) hasta agotar las páginas, extrayendo los datos de cada fila con BeautifulSoup.

```
import csv
import os
import requests
from bs4 import BeautifulSoup

BASE_URL = "http://localhost:3000"
HEADERS = {"User-Agent": "MineriaWeb-2026-2/1.0 (+scraper-tienda-virtual)"}

def get_soup(url: str, parser: str = "html.parser") -> BeautifulSoup:
    respuesta = requests.get(url, headers=HEADERS, timeout=10)
    respuesta.raise_for_status()
    return BeautifulSoup(respuesta.content, parser)
```

#### 1. Descubrir la URL del listado de clientes a partir del sitemap

```
sitemap_index = get_soup(f"{BASE_URL}/sitemap.xml", "xml")
sub_sitemaps = [loc.text for loc in sitemap_index.find_all("loc")]
clientes_sitemap_url = next(url for url in sub_sitemaps if "sitemap-clientes" in url)

sitemap_clientes = get_soup(clientes_sitemap_url, "xml")
urls_clientes_sitemap = [loc.text for loc in sitemap_clientes.find_all("loc")]
print("URLs declaradas en sitemap-clientes.xml:", urls_clientes_sitemap)

clientes_base_url = next(url for url in urls_clientes_sitemap if
url.rstrip("/").endswith("/clientes"))
print("\nListado base de clientes:", clientes_base_url)
```

URLs declaradas en sitemap-clientes.xml: ['http://localhost:3000/clientes', 'http://localhost:3000/clientes/nuevo']

Listado base de clientes: http://localhost:3000/clientes

## 2. Función de paginación

El componente `Pagination` de la tienda virtual deshabilita el enlace **Siguiente** (lo convierte de `<a>` a `<span>`) cuando ya no hay más páginas. Aprovechamos esa señal del HTML para saber cuándo detenernos, sin tener que calcular el total de páginas de antemano.

```
def hay_pagina_siguiente(soup: BeautifulSoup) -> bool:
    nav = soup.select_one('nav[aria-label="Paginacion"]')
    if nav is None:
        return False
    return any(enlace.get_text(strip=True) == "Siguiente" for enlace in
nav.select("a"))
```

## 3. Scraping de cada fila de la tabla de clientes

```
def scrape_fila_cliente(fila) -> dict:
    celdas = fila.find_all("td")
    return {
        "id": fila["data-cliente-id"],
        "dni": fila["data-dni"],
        "nombre": celdas[0].get_text(strip=True),
        "apellidos": celdas[1].get_text(strip=True),
        "email": celdas[3].get_text(strip=True),
        "ciudad": fila["data-ciudad"],
        "pais": fila["data-pais"],
    }
```

```
clientes = []
numero_pagina = 1

while True:
    soup = get_soup(f"{clientes_base_url}?page={numero_pagina}")
    filas = soup.select("tr[data-cliente-id]")
    if not filas:
        break

    clientes.extend(scrape_fila_cliente(fila) for fila in filas)
    print(f"Pagina {numero_pagina}: {len(filas)} clientes")

    if not hay_pagina_siguiente(soup):
        break
    numero_pagina += 1

print(f"\nTotal de clientes scrapeados: {len(clientes)} en {numero_pagina} paginas")
```

Total de clientes scrapeados: 0 en 1 paginas

## 4. Guardar los datos en `data/clientes.csv`

```
DATA_DIR = os.path.join(os.getcwd(), "..", "data")
os.makedirs(DATA_DIR, exist_ok=True)
OUTPUT_PATH = os.path.join(DATA_DIR, "clientes.csv")

with open(OUTPUT_PATH, mode="w", newline="", encoding="utf-8") as archivo:
```

```
writer = csv.DictWriter(archivo, fieldnames=clientes[0].keys())
writer.writeheader()
writer.writerows(clientes)

print(f"Se guardaron {len(clientes)} clientes en {OUTPUT_PATH}")
```

```
-----
IndexError                                Traceback (most recent call last)
Cell In[6], line 6
      2 os.makedirs(DATA_DIR, exist_ok=True)
      3 OUTPUT_PATH = os.path.join(DATA_DIR, "clientes.csv")
      4
      5 with open(OUTPUT_PATH, mode="w", newline="", encoding="utf-8") as archivo:
----> 6     writer = csv.DictWriter(archivo, fieldnames=clientes[0].keys())
      7     writer.writeheader()
      8     writer.writerows(clientes)
      9
```

```
IndexError: list index out of range
```